



TRAINING HANDBOOK · DATA ENGINEERING TRACK

Azure Databricks Workflows

A focused guide to orchestration on Azure Databricks — what Workflows are, their key components, the benefits over hand-running or external schedulers, and how to deploy production workloads through the UI and as code. Threaded through the **SkyOps Analytics** fleet pipeline.

4

Topics

Job

Task · DAG

1

Hands-on lab

2026

Current tooling

What you will learn

A single, focused module covering all four topics on Azure Databricks Workflows — with detailed notes, diagrams, illustrative UI walkthroughs, runnable configuration and a hands-on deployment lab. Terminology reflects the 2026 landscape: Lakeflow Jobs, Unity Catalog and Declarative Automation Bundles.

1 What are Azure Databricks Workflows?

\$1

The built-in orchestration capability and the problem it solves

2 Key components

\$2

Jobs, tasks, the DAG, triggers, task types, compute and parameters

3 Benefits

\$3

Reliability, cost, observability, governance · Workflows vs external orchestrators

4 Deploying workloads with Workflows

\$4

The UI, Declarative Automation Bundles, CI/CD across environments, monitoring

01

ORCHESTRATION

Azure Databricks Workflows

The built-in way to turn notebooks, SQL and pipelines into reliable, scheduled, monitored production workloads — what they are, their components, their benefits, and how to deploy with them.

Azure Databricks Workflows

A notebook you run by hand is a prototype. A notebook that runs itself on schedule, recovers from failure and tells you when something breaks is a **production workload**.

Azure Databricks Workflows is the service that makes that leap — and it is built into the platform, so there is no separate orchestrator to install or operate. We use the **SkyOps Analytics** aircraft-fleet pipeline as the running example throughout.

Learning objectives

- Explain what Azure Databricks Workflows is and the problem it solves
- Identify the key components: jobs, tasks, the DAG and triggers
- Describe the main benefits over hand-running or external orchestrators
- Deploy a workload through the UI and as code
- Use Declarative Automation Bundles for CI/CD across environments
- Schedule, parameterise and monitor a deployed workload

1 What are Azure Databricks Workflows?

Workflows is the fully-managed orchestration capability of Azure Databricks. It lets you coordinate multiple steps — notebooks, scripts, SQL, pipelines — into a single, automated **job** that runs on a schedule or in response to an event, with dependencies, retries, alerting and a full run history. Because it is part of the platform, you do not stand up or patch a separate scheduler; the same workspace that authors the code also runs it in production.

A note on names

You will see three terms for the same thing. **Workflows** is the overall capability. **Jobs** is the resource you create (now under **Jobs & Pipelines** in the sidebar). **Lakeflow Jobs** is the current product branding. They refer to the same orchestration engine — this handbook uses “Workflows” for the capability and “job” for the thing you build.

Conceptually, Workflows sits *above* the assets you build and *below* the data they produce. It orchestrates your work, runs it on managed compute, keeps it inside Unity Catalog governance, and lands the results as Delta tables on the lakehouse.

Where Workflows sits — the orchestration layer of the Databricks platform

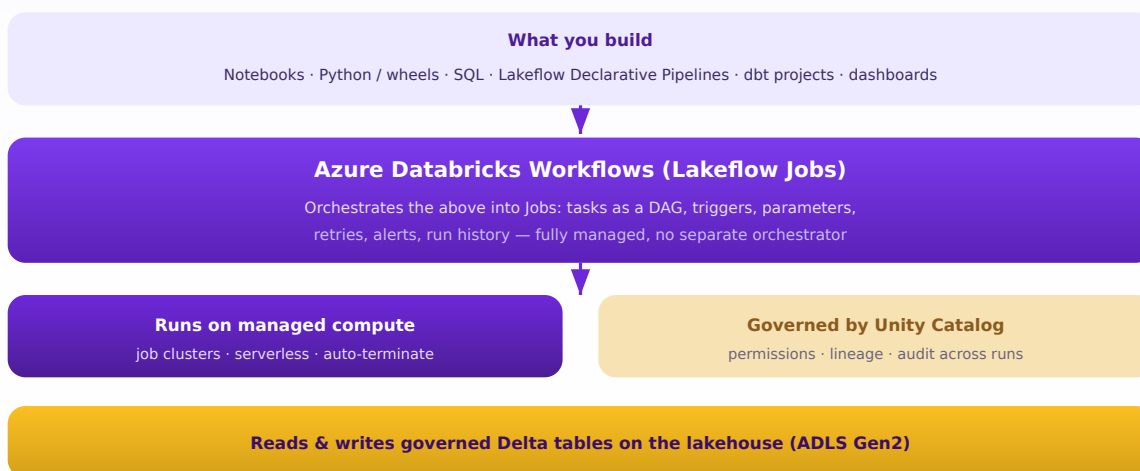


Figure 1.1. Workflows is the built-in orchestration layer: it turns the assets you build into managed Jobs, runs them on managed compute under Unity Catalog governance, and lands results on the lakehouse.

✈ Why SkyOps needs it

Our fleet pipeline has four steps that must run in order every night: ingest raw telemetry, clean it, build the KPI table, refresh the dashboard. Running those by hand is fragile and unobservable. As a Workflow they become one job that runs at 02:00, retries transient failures, and emails the team if anything breaks.

2 Key components

The model has just three core ideas. A **Job** is the unit you schedule and run. It contains one or more **Tasks**, arranged as a **Directed Acyclic Graph (DAG)** so dependencies are explicit and order is guaranteed. A **Trigger** decides when the job runs. Everything else is configuration on those three.

A Lakeflow Job — tasks as a DAG, with trigger, compute & alerts

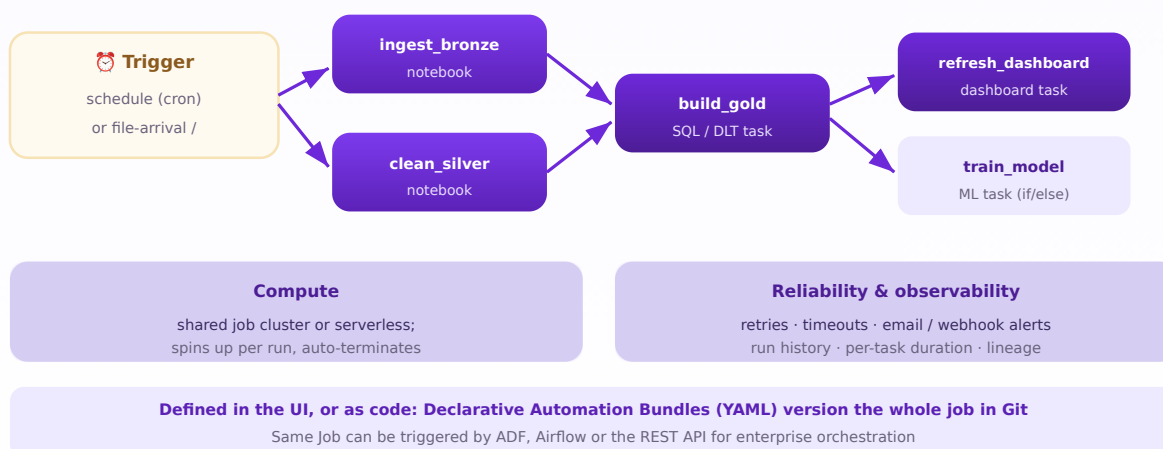


Figure 3.1. A Lakeflow Job models work as a DAG of tasks with dependencies, started by a trigger, run on managed compute, with built-in retries, alerts and run history.

Each **task** declares a **type**. The common ones are below; tasks can also branch and loop, so a single job can express real control flow.

Task type	Runs
Notebook	A Databricks notebook — the most common task
Python script / wheel	A <code>.py</code> file or packaged library entry point
SQL	A query, alert or dashboard on a SQL warehouse
Pipeline	A Lakeflow Declarative Pipeline (formerly Delta Live Tables)
dbt	A dbt project's models
Run job	Another job — compose larger workflows from smaller ones
If/else · For each	Control-flow tasks for branching and looping

https://adb-skyops.azuredatabricks.net/jobs/create

Jobs & Pipelines ► Create job ► add task skyops_nightly_etl

Task name: clean_silver

Compute: Serverless ▼ (or job cluster)

Type: Notebook ▼

Parameters: {"run_date": "{{job.start_time}}"}

Path: /Repos/skyops/02_build_silver

Retries: 2 · 5-min interval

Depends on: ingest_bronze ▼

Notifications: email on failure ▼

Create task Add another task Add Git settings

Screenshot 2.1. Configuring a single task: its type and source, the task it depends on, the compute it runs on, run-time parameters, retries and notifications — the building blocks of a Job.

A task runs on **compute** — a job cluster created for the run, a shared job cluster across tasks, or **serverless**. **Triggers** start the job: a **schedule** (Quartz cron), a **file-arrival** trigger (new data lands in storage), a **table-update** trigger (an upstream Delta table changes), **continuous** mode, or on-demand via the UI, REST API, CLI, ADF or Airflow. Reliability settings — **retries**, **timeouts**, **notifications** and **concurrency limits** — round out the configuration.

💡 Pass values between tasks

Job-level **parameters** are pushed to every task, and a task can publish **task values** (`dbutils.jobs.taskValues`) that downstream tasks read — so step two can act on what step one computed without writing to a side table.

3 Benefits

Why use Workflows rather than hand-running notebooks or wiring up an external scheduler?

Benefit	What it means in practice
Built-in & managed	No separate orchestrator to deploy, scale or patch — it is part of the workspace
Reliable	Automatic retries, timeouts, failure alerts, and <i>Repair run</i> to re-run only what failed
Cost-efficient	Job / serverless compute spins up per run and auto-terminates; tasks can share one cluster
Observable	Run history, per-task durations, a visual DAG, and data lineage in Unity Catalog
Governed	Unity Catalog permissions and a defined <i>run-as</i> identity apply to every run
Flexible	Many task types plus if/else and for-each let one job express a real pipeline
As-code	Whole jobs version in Git via Declarative Automation Bundles, enabling CI/CD

The screenshot shows the Lakeflow job interface for a job named `skyops_nightly_etl`. The URL is `https://adb-skyops.azuredatabricks.net/jobs/1043`. The interface includes a header with the job name and a cron schedule `0 0 2 * * ? (02:00 daily)`. Below the header, there is a section for "Recent runs" with a table showing the last three runs. To the right of the table is a "Task graph" showing a sequence of tasks: `ingest_bronze`, `clean_silver`, `build_gold`, and `refresh_dashboard`.

Run	Start	Duration	Status
#1043	02:00 · today	6m 12s	Succeeded
#1042	02:00 · yesterday	6m 02s	Succeeded
#1041	02:00 · 2d ago	—	Failed · retried ✓

Retries: 2 · Timeout: 30m · Alerts: email on failure → data-eng@skyops

Screenshot 3.1. A Lakeflow Job: the run history with status and durations on the left (note the auto-retried failure), the task DAG on the right, and the cron schedule in the header.

Workflows vs an external orchestrator

Use **Workflows** for pipelines that live mostly inside Databricks — it is simpler, cheaper to operate and natively aware of clusters, tables and lineage. Reach for **Airflow** or **Azure Data Factory** when you must orchestrate across many systems beyond Databricks; both can still trigger a Databricks job as one step, so the two approaches compose rather than compete.

4 Deploying workloads with Workflows

There are two ways to deploy a workload, and mature teams use both: the **UI** for authoring and quick iteration, and **code** for anything that reaches production.

Path A — the UI. Under **Jobs & Pipelines**, click *Create job*, add a task per step (as in 2.1), wire up dependencies, choose compute, set a schedule, add parameters and notifications, then run. Fast to build, but a click-built job is hard to review, reproduce or roll back.

Path B — as code. Define the job declaratively with a **Declarative Automation Bundle** (DAB — formerly *Databricks Asset Bundles*; the rename in early 2026 is non-breaking, same `databricks.yml` and CLI). A bundle describes the job, its tasks, schedule and compute as YAML you keep in Git and promote across **targets** (dev, staging, prod).

Deploying a workload with Workflows — develop once, promote across environments

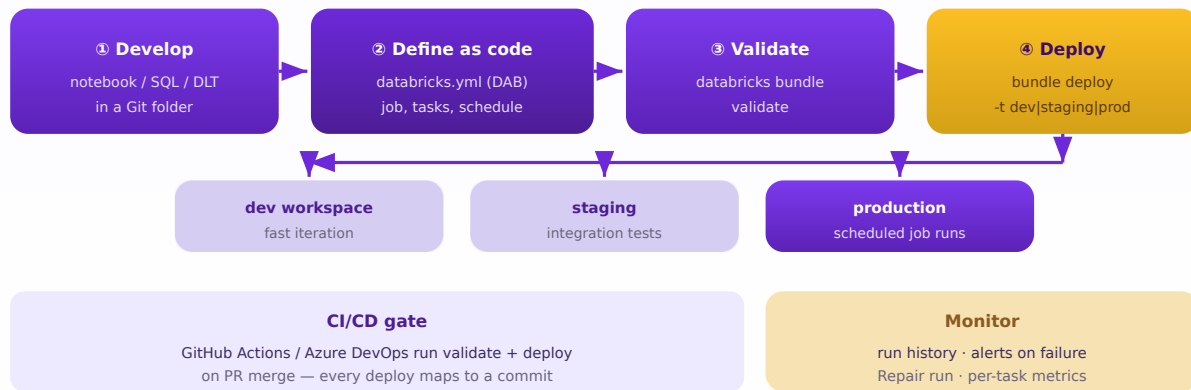


Figure 4.1. A production-grade deployment path: develop in Git, define the job as a Declarative Automation Bundle, validate, then deploy the same definition across dev, staging and production through CI/CD.

YAML

```
# databricks.yml — one workload, promoted across environments
bundle:
  name: skyops_nightly_etl

targets:
  dev:  { default: true, workspace: { host: https://adb-dev.azuredatabricks.net } }
  prod: { workspace: { host: https://adb-prod.azuredatabricks.net } }

resources:
  jobs:
    skyops_nightly_etl:
      name: skyops_nightly_etl
      schedule:
        quartz_cron_expression: "0 0 2 * * ?"      # 02:00 daily
        timezone_id: "Australia/Sydney"
      email_notifications: { on_failure: ["data-eng@skyops.example"] }
      tasks:
        - task_key: ingest_bronze
          notebook_task: { notebook_path: ./01_ingest_telemetry }
        - task_key: clean_silver
          depends_on: [{ task_key: ingest_bronze }]
          notebook_task: { notebook_path: ./02_build_silver }
        - task_key: build_gold
          depends_on: [{ task_key: clean_silver }]
          notebook_task: { notebook_path: ./03_build_fleet_kpis }
```

Then validate and deploy from the CLI — or, better, from CI/CD on a merge to main:

SHELL

```
databricks bundle validate          # check the YAML & resources
databricks bundle deploy -t dev     # push the job to the dev workspace
databricks bundle run skyops_nightly_etl -t dev # trigger a run now
# ...after review & tests pass in CI...
databricks bundle deploy -t prod    # promote the identical definition
```

CONSOLE OUTPUT

```
Validation OK!
Deploying resources...
Updating deployment state...
Deployment complete!
job skyops_nightly_etl → https://adb-dev.azuredatabricks.net/jobs/1043
```

⚠️ Make deployed tasks idempotent

Because runs retry and repair, a task must produce the same result if it runs twice. Prefer `MERGE` or overwrite-by-partition over blind `append` so a retry never double-writes the same telemetry batch.

💡 Monitor what you deploy

After deployment, the job's **runs** page is your operations view — status, durations, the task DAG and logs. Wire **failure alerts** to email or a Slack/Teams webhook, and use **Repair run** to recover a partial failure without repeating successful upstream tasks.

🔧 Lab — Deploy the SkyOps nightly workload

(1) In the UI, create `skyops_nightly_etl` with two notebook tasks where `clean_silver` depends on `ingest_bronze`; set serverless compute, a 2 AM schedule, 2 retries and an email alert. Run it once and open the run's DAG. (2) Now express the same job as a `databricks.yml` bundle with `dev` and `prod` targets. Run `bundle validate` then `deploy -t dev`. (3) Force the second task to fail, then use **Repair run** to re-run only that task. Note which path — UI or bundle — you would trust for a production release, and why.

🔑 Key takeaways

- Workflows is the built-in, managed orchestration of Azure Databricks — no separate scheduler to run
- The model is simple: a Job runs Tasks arranged as a DAG, started by a Trigger
- Tasks span notebooks, Python, SQL, pipelines, dbt and run-job, plus if/else and for-each control flow
- Benefits: reliability, cost-efficient managed compute, observability, governance, and as-code CI/CD
- Deploy via the UI for iteration; via Declarative Automation Bundles (formerly Asset Bundles) for production
- Promote one definition across dev/staging/prod targets, schedule with cron or events, and design tasks to be idempotent